VERONICA: A Runtime Containment Layer for Compositional Resource Amplification in LLM Agent Systems

Keita Amo
Independent Researcher
https://github.com/amabito/veronica-core

Abstract

Large language model (LLM) agent frameworks expose a class of runaway failure modes that differ qualitatively from classical software defects. Retry amplification, recursive tool invocation loops, multi-agent cost cascades, and WebSocket session runaways share a common structure: they originate from individually reasonable local decisions that compose into unbounded global resource consumption. Existing frameworks (LangChain, AG2, LangGraph) delegate containment to application developers through per-call timeouts and manual budget checks, leaving compositional failures unaddressed.

We present VERONICA, a runtime containment layer that enforces chain-level resource bounds without requiring modifications to agent logic. VERONICA introduces four primitives—BudgetEnforcer, AgentStepGuard, CircuitBreaker, and RetryContainer—unified under a RuntimePolicy structural protocol and composed by PolicyPipeline with first-denial-wins semantics. A chain-scoped ExecutionContext enforces simultaneous bounds on cost, step count, retry count, and wall-clock time across all LLM and tool calls within a single agent run, with propagation through ASGI/WSGI middleware and WebSocket sessions.

We state six safety guarantees (G1–G6) with code-grounded proofs, present benchmark results demonstrating 53–90% operation reduction versus uncontained baselines, and measure a policy pipeline overhead of 11.43 microseconds per call. VERONICA is available as `veronica-core` on PyPI (v2.0.0, 3214 tests, 92% coverage).

1 Introduction

LLM agent systems execute multi-step plans by repeatedly calling language models and invoking tools based on model outputs. This structure creates several runaway failure modes that are absent in classical request/response software:

Retry amplification. When L nested tool layers each retry r times on failure, a single user action generates at most $(1+r)^L$ LLM calls. At L=3, r=3, this is 64 calls from a single user action. Frameworks that implement per-call retry budgets do not track cross-layer retry counts; the ceiling is the product of all per-layer budgets.

Recursive tool loops. An agent producing a tool call whose output triggers the same tool call will loop indefinitely unless an external step counter intervenes. LangChain's `max_iterations` parameter addresses this within a single chain but does not apply across chain boundaries in multi-agent topologies.

Multi-agent amplification. A coordinator spawning K sub-agents in a tree of depth D creates $(K^{D+1}-1)/(K-1)$ total agents; at K=5, D=2, that is 31 agents. If each agent independently makes S steps with r retries, total calls are $31 \times S \times (1+r)$. The frameworks surveyed in Section 2 do not enforce a cross-agent cost ceiling without custom application code.

WebSocket runaway. Long-running WebSocket sessions that stream LLM output accumulate cost and step counts across hundreds of receive/send cycles. Existing ASGI frameworks provide no budget enforcement for WebSocket scopes.

These failure modes are compositional: they arise from combining features (retry logic, agent spawning, tool invocation) that are individually reasonable but collectively unbounded. Classical per-call defenses are insufficient because each call is legitimate; only the aggregate violates the resource constraint.

VERONICA addresses these failure modes through a single architectural decision: containment is enforced at the chain level rather than the call level. Each agent run or HTTP request receives a dedicated `ExecutionContext` that tracks cumulative cost, step count, and retry count across all operations, enforcing hard limits without requiring agent-level awareness of the containment layer.

1.1 Contributions

1. A formal model of compositional resource amplification in LLM agent systems, covering retry, recursive tool, and multi-agent failure modes (Section 3).
2. A runtime containment architecture with four composable policy primitives and a chain-scoped execution context (Section 4).
3. Six safety guarantees (G1–G6) with proofs grounded in implementation code paths (Section 5).
4. Empirical evaluation against four baseline configurations showing 53–90% operation reduction with sub-12-microsecond overhead per call (Section 6).

The system is available as `veronica-core` on PyPI (v2.0.0, 3214 tests, 92% coverage). All benchmarks and tests are included in the repository.

2 Background and Related Work

2.1 LLM Agent Frameworks

LangChain [1] provides `max_iterations` for chain step limits and per-call retry logic through `tenacity`-backed decorators. It does not enforce cross-chain cost ceilings, multi-agent retry budgets, or WebSocket containment.

AG2 (AutoGen) [2] supports nested agent conversations with configurable `max_consecutive_auto_reply`. Resource limits are delegated to application code. No circuit breaker or distributed budget enforcement is provided.

LangGraph [3] provides graph-based agent orchestration with checkpoint support and step-level event streams. Budget enforcement is not a first-class primitive; users must instrument graph nodes manually.

CrewAI [4] provides role-based agent teams with task delegation. Cost tracking is not provided at the framework level.

2.2 Tool-Augmented and Autonomous Agents

ReAct [5] interleaves reasoning and tool invocation, creating multi-step execution traces where each tool call may trigger further reasoning. ReAct does not bound the total number of reasoning-action cycles; unbounded loops are possible when the model repeatedly selects the same tool.

Toolformer [6] fine-tunes LLMs to insert API calls into generated text. Cost containment is delegated to the API layer; the model itself has no budget awareness.

AutoGPT [7] chains LLM calls autonomously with minimal human oversight. Early deployments demonstrated unbounded cost accumulation due to recursive goal decomposition, motivating external containment mechanisms.

2.3 Containment and Safety Libraries

Guardrails AI [8] focuses on output validation: schema enforcement, content filtering, and structured output parsing. It does not address resource consumption during execution.

NeMo Guardrails [9] provides dialog flow control and safety filtering through conversation rail specifications. Retry and cost containment are outside scope.

OpenAI function calling with `tool_choice` parameters constrains which tools the model selects but does not bound total cost or retry depth.

Agent safety. Ruan et al. [10] identify risk categories for LLM agents including resource exhaustion and uncontrolled tool invocation, but propose monitoring rather than enforcement mechanisms. VERONICA addresses the enforcement gap.

To our knowledge, none of the above systems enforce chain-level cost ceilings, cross-layer retry budgets, or circuit breaking at the framework abstraction level.

2.4 Resilience Patterns in Distributed Systems

The circuit breaker pattern [11] is standard in distributed systems for failure isolation. Libraries such as Hystrix [12], resilience4j [13], and Polly [14] implement circuit breakers, bulkheads, and retry policies for microservice architectures. These libraries target service-to-service communication; they do not model the compositional amplification structure of LLM agent systems (Section 3), where retry layers multiply rather than add. VERONICA adapts the circuit breaker pattern to the LLM call domain and extends it with chain-level budget enforcement that spans multiple retry and agent layers.

3 Amplification Model

3.1 Definitions

Definition 1 (Call Node) *A call node v is a single LLM API call or tool invocation with attributes:* $\text{cost}(v) \geq 0$ *USD,* $\text{tokens_in}(v)$, $\text{tokens_out}(v)$, $\text{retries}(v) \geq 0$.

Definition 2 (Agent Chain) *An agent chain C is a sequence of call nodes $v_1, v_2, \ldots, v_n$ with:*

$$total_cost(C) = \sum_{i=1}^{n} \text{cost}(v_i) \tag{1}$$

$$total_steps(C) = n \tag{2}$$

$$total_retries(C) = \sum_{i=1}^{n} \text{retries}(v_i) \tag{3}$$

Definition 3 (Amplification Factor) *For a chain C with known minimum required steps $min_steps(C)$:*

$$A(C) = \frac{total_steps(C)}{min_steps(C)}$$

In pathological cases (retry explosion, recursive loops), $A(C)$ is unbounded.

3.2 Multi-Layer Retry Amplification

Theorem 1 (Multi-Layer Retry) *For a nested call structure with L layers, each independently retrying with r_i retries at layer i, the worst-case total calls are:*

$$Total_calls = \prod_{i=1}^{L}(1 + r_i)$$

Proof. By induction on L. For L=1: at most $1 + r_1$ calls. Inductively, each call at layer k triggers at most $\prod_{j=k+1}^{L}(1 + r_j)$ calls in deeper layers; summing over at most $(1 + r_k)$ attempts at layer k gives the product. QED. □

Example. At L=3, r_i=3 for all i: Total_calls = $4^3 = 64$. At \$0.01/call, a single user action incurs \$0.64 in the worst case versus \$0.01 for the minimum path.

3.3 Multi-Agent Amplification

Theorem 2 (Multi-Agent Amplification) *For a coordinator spawning K sub-agents to depth D, each making S steps with r retries:*

$$Total_calls = \frac{K^{D+1} - 1}{K - 1} \cdot S \cdot (1 + r)$$

Proof sketch. The total number of agents in a K-ary tree of depth D is the geometric series $\sum_{d=0}^{D} K^d = (K^{D+1} - 1)/(K - 1)$. Each agent independently executes S steps with up to r retries per step, contributing $S \cdot (1 + r)$ calls. The product gives the total. This assumes no cross-agent coordination on retry budgets (the scenario VERONICA is designed to contain). QED. □

Example. At K=5, D=2, S=10, r=3: Total_calls = $31 \times 10 \times 4 = 1{,}240$.

3.4 Failure Model

VERONICA's safety guarantees (Section 5) assume the following failure model. Deviations from these assumptions weaken specific guarantees as noted.

Process-local mode (default). All state—budget accumulators, step counters, circuit breaker state machines—resides in the process address space, protected by `threading.Lock`. The failure model is:

- **Thread crash.** A thread holding a lock that terminates abnormally (e.g., via `KeyboardInterrupt` between lock acquisition and release) may leave the lock in an inconsistent state. Python's `threading.Lock` is not reentrant-safe across `KeyboardInterrupt`. In practice, `with self._lock:` blocks are short (microseconds), minimizing the vulnerability window. This is a known limitation of CPython's `threading` module and affects all lock-based Python libraries.
- **Process crash.** All in-memory state is lost. No durability guarantee is provided for process-local mode. Callers requiring crash recovery should use the Redis backend.

Distributed mode (Redis backend). Budget and circuit breaker state is stored in Redis. The failure model extends to:

- **Network partition.** If the application cannot reach Redis, behavior depends on `fallback_on_error`:

  - `fallback_on_error=True` (default): The system falls back to a process-local budget backend seeded with the last known Redis state. G1 (cost bound) holds within each process but not across processes during the partition. On reconnection, `_reconcile_with_redis()` pushes the locally accumulated delta back to Redis.
  - `fallback_on_error=False`: The call raises `RedisError`, and the caller must handle it. G1 is not weakened because no cost is committed.
- **Partial commit.** Redis `INCRBYFLOAT` is atomic for a single key. Budget commit is a single `INCRBYFLOAT` operation; there is no multi-key transaction that could be partially committed. Circuit breaker state transitions use Lua scripts executed via `eval()`, which Redis guarantees to run atomically.
- **Redis atomicity scope.** Lua scripts provide atomicity within a single Redis instance. In Redis Cluster mode, keys must hash to the same slot for cross-key atomicity; VERONICA uses `{chain_id}` hash tags to ensure co-location. Cluster failover (replica promotion) may lose the most recent write if the primary fails before replication completes; this is a Redis Cluster property, not a VERONICA limitation.
- **Clock skew.** `CircuitBreaker` recovery timeout and `SharedTimeoutPool` deadlines use `time.time()` (wall clock). On the local host, monotonic alternatives (`time.monotonic()`) would avoid NTP adjustment issues; the current implementation uses `time.time()` for Redis compatibility, where server-side `TIME` is the only available clock. Clock skew between application hosts and Redis does not affect G4 (failure isolation) because the Lua scripts compare timestamps stored and read within the same Redis instance.

3.5 VERONICA Containment Bounds

Theorem 3 (Cost Bound) *For any agent chain X under an `ExecutionContext` with `max_cost_usd` $= B$: $total_cost(X) \leq B$, independent of agent count, retry depth, or topology.*

Theorem 4 (Step Bound) *For any agent chain X under an `ExecutionContext` with `max_steps` $= S$: $total_steps(X) \leq S$.*

Theorem 5 (Retry Bound) *For any agent chain X under an `ExecutionContext` with `max_retries_total` $= R$: $total_retries(X) \leq R$, bounding total calls at $1 + R$.*

Proofs are given in Section 5 (G1–G3). The bounds hold simultaneously and independently; violation of any triggers `Decision.HALT` for all subsequent calls.

4 System Architecture

4.1 Overview

```
User / Framework
      |
      v
 [ExecutionContext]  <-- chain-level containment boundary
      |
      +-- [PolicyPipeline]
      |       |
      |       +-- [BudgetEnforcer]     cost_usd ceiling
      |       +-- [AgentStepGuard]     max_steps limit
```

```
   |       +-- [CircuitBreaker]     failure isolation
   |       +-- [RetryContainer]     retry budget
   |
   +-- [ExecutionGraph]             call-tree observation
   |
   +-- [CancellationToken]          cooperative shutdown
   |
   +-- [SharedTimeoutPool]          wall-clock timeout
   |
   v
LLM API / Tools
```

The containment layer sits between the agent framework and the LLM API. Each `wrap_llm_call()` or `wrap_tool_call()` invocation passes through the policy pipeline before dispatching to the underlying callable.

4.2 RuntimePolicy Protocol

All policy primitives implement the `RuntimePolicy` structural protocol (`src/veronica_core/runtime_policy.py`):

```python
@runtime_checkable
class RuntimePolicy(Protocol):
    def check(self, context: PolicyContext) -> PolicyDecision: ...
    def reset(self) -> None: ...

    @property
    def policy_type(self) -> str: ...
```

`PolicyContext` carries ambient information about the current operation:

```python
@dataclass
class PolicyContext:
    cost_usd: float = 0.0
    step_count: int = 0
    entity_id: str = ""
    chain_id: str = ""
    timestamp: float = field(default_factory=time.time)
    metadata: dict = field(default_factory=dict)
```

`PolicyDecision` carries the allow/deny result, policy identity, and optional degradation actions (model downgrade, rate limiting):

```python
@dataclass
class PolicyDecision:
    allowed: bool
    policy_type: str
    reason: str = ""
    partial_result: Any = None
    degradation_action: str | None = None
    fallback_model: str | None = None
    rate_limit_ms: int = 0
```

The `degradation_action` field enables graceful degradation responses (model downgrade, context trimming, rate limiting) as alternatives to hard denial, allowing callers to adapt behavior without necessarily halting.

4.3 PolicyPipeline

`PolicyPipeline` (`src/veronica_core/runtime_policy.py`) composes multiple `RuntimePolicy` instances with AND semantics: the first denial terminates evaluation and returns the denying decision.

```
class PolicyPipeline:
    def evaluate(self, context: PolicyContext) -> PolicyDecision:
        for policy in self._policies:
            decision = policy.check(context)
            if not decision.allowed:
                return decision # First denial wins
        return PolicyDecision(allowed=True, policy_type="pipeline", ...)
```

No override mechanism exists. This is a deliberate design decision: safety policies must not be overridable by lower-priority policies or application code.

4.4 BudgetEnforcer

`BudgetEnforcer` (`src/veronica_core/budget.py`) enforces a USD ceiling with thread-safe check-before-commit semantics:

```
def spend(self, amount_usd: float) -> bool:
    with self._lock:
        projected = self._spent_usd + amount_usd
        if projected > self.limit_usd:
            self._exceeded = True
            return False
        self._spent_usd = projected
        self._call_count += 1
        return True
```

Input validation at both construction (`__post_init__`) and call time (`spend`) rejects NaN, Inf, and negative amounts with `ValueError`, preventing cost poisoning attacks that could corrupt the accumulator via IEEE-754 arithmetic anomalies.

Measured overhead: 0.191 microseconds per `spend()` call (100,000 calls).

4.5 AgentStepGuard

`AgentStepGuard` (`src/veronica_core/agent_guard.py`) limits agent iterations via increment-before-check protocol:

```
def step(self, result: Any = None) -> bool:
    with self._lock:
        self._current_step += 1     # increment first
        if result is not None:
            self._last_result = result # preserve partial result
        if self._current_step >= self.max_steps:
            return False            # then check
    return True
```

The increment-before-check ordering prevents the off-by-one error where `max_steps=25` would allow 26 steps if check-before-increment were used. The `_last_result` field preserves the most recent partial output for halted chains.

4.6 CircuitBreaker

`CircuitBreaker` (`src/veronica_core/circuit_breaker.py`) implements the three-state circuit breaker pattern with atomic state transitions via `threading.Lock`. States:

- **CLOSED**: consecutive failures < threshold; all requests allowed
- **OPEN**: failures ≥ threshold; all requests denied
- **HALF_OPEN**: recovery timeout elapsed; exactly one test request allowed

The HALF_OPEN single-slot constraint, enforced by `_half_open_in_flight`, prevents thundering-herd on recovery: concurrent callers in HALF_OPEN all receive `allowed=False` except the first. This is identical in semantics to the original circuit breaker pattern [11] and prevents load amplification on an already-stressed endpoint.

`bind_to_context()` prevents accidental sharing of a `CircuitBreaker` instance across independent chains, which would corrupt failure counts.

In distributed deployments (`src/veronica_core/distributed.py`), state transitions use Lua scripts executed atomically by Redis `eval()`, extending the consistency guarantee to multi-process environments without application-level locking.

Measured overhead: 0.528 microseconds per `check()` call (100,000 calls, CLOSED state).

4.7 RetryContainer

`RetryContainer` (`src/veronica_core/retry.py`) enforces a chain-wide retry budget with exponential backoff and jitter:

```
def execute(self, fn: Callable[..., T], *args, **kwargs) -> T:
    for attempt in range(self.max_retries + 1):
        try:
            return fn(*args, **kwargs)
        except Exception as e:
            if attempt == self.max_retries:
                raise
            delay = min(self.backoff_base * (2 ** attempt), self.backoff_max)
            delay *= 1.0 + self.jitter * (2 * random.random() - 1)
            time.sleep(delay)
```

Jitter (default ±25%) prevents thundering-herd when multiple `RetryContainer` instances retry simultaneously after a shared service failure. The serialization lock held during `fn()` execution enforces serial retry semantics, preventing concurrent retries from racing on the attempt counter; this is a deliberate design decision, not a performance issue.

4.8 ExecutionContext

`ExecutionContext` (`src/veronica_core/containment/execution_context.py`) is the chain-level container. `ExecutionConfig` specifies hard limits:

```
@dataclass(frozen=True)
class ExecutionConfig:
    max_cost_usd: float
    max_steps: int
    max_retries_total: int
    timeout_ms: int = 0
```

All limits are validated at construction to be finite and non-negative. The context is used as a context manager:

```
config = ExecutionConfig(
    max_cost_usd=1.0, max_steps=50,
    max_retries_total=10, timeout_ms=30_000
)
with ExecutionContext(config=config, pipeline=pipeline) as ctx:
    decision = ctx.wrap_llm_call(fn=lambda: client.chat(...))
    if decision == Decision.HALT:
        break
```

On entering the context manager, the wall-clock timeout is registered with `SharedTimeoutPool` (a singleton daemon-thread scheduler with a heap-priority queue of deadline-callback pairs). When the deadline fires, `CancellationToken.cancel()` is called, and all subsequent `wrap_llm_call()` invocations return `Decision.HALT` without dispatching.

Measured overhead: 11.43 microseconds per `wrap_llm_call()` call over a full `ExecutionContext` round-trip (10,000 calls).

4.9 Middleware Integration

`VeronicaASGIMiddleware` and `VeronicaWSGIMiddleware` (`src/veronica_core/middleware.py`) create a fresh `ExecutionContext` per HTTP request, stored in a `ContextVar` accessible via `get_current_execution_co` For WebSocket scopes, each `receive()` and `send()` call increments the step counter. Budget exceeded at pre-flight returns HTTP 429; mid-session limit triggers `websocket.close` code 1008.

Integration with LangChain, AG2, LangGraph, LlamaIndex, CrewAI, and ROS2 is provided through framework-specific adapters in `src/veronica_core/adapters/`.

4.10 ExecutionGraph

`ExecutionGraph` (`src/veronica_core/containment/execution_graph.py`) tracks the parent-child call tree for observation and diagnostics. Each LLM and tool call creates a `Node` with timing, cost, and token counts. The graph supports divergence heuristics based on repeated (kind, name) node signatures, enabling early detection of pathological loops before they exhaust the budget.

5 Formal Safety Guarantees

We state six safety guarantees and identify the code paths that enforce each. All guarantees assume a correctly configured `ExecutionContext` with finite, non-negative limit values; G6 proves that invalid configurations cannot be created.

5.1 G1: Cost Bound

Guarantee. For any agent chain under an `ExecutionContext` with `ExecutionConfig.max_cost_usd` $= B$:

$$\text{total_cost}(\text{chain}) \leq B$$

Proof. The budget accumulator is `_cost_usd_accumulated` in `ExecutionContext`. Every LLM call passes through `_wrap()`, which calls `_commit_cost(actual_cost)` after the callable returns:

```
with self._lock:
    projected = self._cost_usd_accumulated + actual_cost
    if projected > self._config.max_cost_usd:
```

```
        self._aborted = True
        self._abort_reason = "budget_exceeded"
        return Decision.HALT
    self._cost_usd_accumulated = projected
```

No code path commits cost without first checking the ceiling. The lock prevents concurrent commits from racing past the ceiling. Additionally, `BudgetEnforcer.spend()` (`src/veronica_core/budget.py`) uses the identical pattern for standalone use.

Input validation in `BudgetEnforcer.__post_init__()` and `spend()` rejects NaN, Inf, and negative values, preventing cost poisoning via IEEE-754 arithmetic anomalies.

In distributed mode (`RedisBudgetBackend`, `src/veronica_core/distributed.py`), Redis `INCRBYFLOAT` is atomic; an epsilon guard (`_BUDGET_EPSILON = 1e-9`) prevents spurious under-enforcement from floating-point rounding.

G1 holds in both local and distributed configurations. QED. □

5.2 G2: Termination Guarantee

Guarantee. For any agent chain under an `ExecutionContext` with `ExecutionConfig.max_steps` $= S$:

$$\text{successful_operations}(\text{chain}) \leq S$$

Proof. The step counter is `_step_count` in `ExecutionContext`. Every call through `wrap_llm_call()` or `wrap_tool_call()` passes through `_wrap()`:

```
with self._lock:
    if self._step_count >= self._config.max_steps:
        self._aborted = True
        return Decision.HALT
```

After a successful call, `_step_count` is incremented. The check-before-increment ordering means: at step count S, the next call is denied before `_step_count` would become $S{+}1$. No code path increments past S. QED. □

5.3 G3: Retry Budget

Guarantee. For any agent chain under an `ExecutionContext` with `ExecutionConfig.max_retries_total` $= R$:

$$\text{total_retries}(\text{chain}) \leq R$$

Proof. The retry counter is `_retries_used` in `ExecutionContext`. Before each retry in `_wrap()`, `_check_retry_budget()` is called:

```
with self._lock:
    if self._retries_used >= self._config.max_retries_total:
        self._aborted = True
        return Decision.HALT
    self._retries_used += 1
```

The check-before-increment protocol provides the same bound as G2. When `RetryContainer` instances operate within an `ExecutionContext`, the chain-level check fires before each retry attempt, enforcing the cross-container ceiling. QED. □

5.4 G4: Failure Isolation

Guarantee. Once `CircuitBreaker` enters OPEN state:

$$\text{state} = \texttt{OPEN} \implies \texttt{check}(ctx).\texttt{allowed} = \texttt{False}$$

during all calls until at least `recovery_timeout` seconds have elapsed.

Proof. State transitions are protected by `threading.Lock`. The only transition out of OPEN is OPEN → HALF_OPEN, gated by `_maybe_half_open_locked()`:

```
time.time() - self._last_failure_time >= self.recovery_timeout
```

evaluated under lock, preventing two threads from simultaneously observing OPEN as expired. No code path transitions OPEN directly to CLOSED; the mandatory HALF_OPEN state ensures at least one test call before recovery.

In distributed mode, Lua scripts executed via Redis `eval()` provide identical atomicity across processes. QED. □

5.5 G5: Wall-Clock Timeout

Guarantee. For any chain under `ExecutionContext` with `timeout_ms` $= T > 0$:

$$\text{elapsed_ms} > T \implies \forall \text{ new calls: } \texttt{wrap_llm_call()} \to \texttt{Decision.HALT}$$

Proof. On `__enter__`, a deadline is registered with `SharedTimeoutPool`. When the deadline fires, `_on_timeout()` calls `CancellationToken.cancel()`. All subsequent `wrap_llm_call()` calls check `CancellationToken.is_cancelled` before dispatching; cancelled contexts return `Decision.HALT` immediately.

Caveat. Operations already in-flight when the timeout fires may complete before observing cancellation. G5 applies only to new operations; in-flight operations must poll `is_cancelled` for preemptive cancellation. QED. □

5.6 G6: Configuration Validity

Guarantee. An `ExecutionContext` with NaN, Inf, or negative limits cannot be created.

Proof. `ExecutionConfig.__post_init__()` validates all fields:

```
if math.isnan(self.max_cost_usd) or math.isinf(self.max_cost_usd):
    raise ValueError(...)
if self.max_cost_usd < 0:
    raise ValueError(...)
# ... analogous for max_steps, max_retries_total, timeout_ms
```

`BudgetEnforcer.__post_init__()`, `CircuitBreaker.__post_init__()`, and `AgentStepGuard` validation analogously reject invalid inputs. Invalid configurations raise `ValueError` at construction, not at runtime. QED. □

5.7 Compositional Safety

Theorem 6 (Compositional Bound) *For a `PolicyPipeline` containing `BudgetEnforcer(limit_usd=B)`, `AgentStepGuard(max_steps=S)`, and `RetryContainer(max_retries=R)`, the following hold simultaneously:*

- $total_cost \leq B$
- $total_steps \leq S$
- $total_retries \leq R$

Proof sketch. Each bound is enforced by an independent check-before-commit protocol under its own lock. Violation of any bound returns `False`/`HALT` before the underlying callable is invoked. `PolicyPipeline.evaluate()` returns the first denial, preventing execution from reaching any primitive with a satisfied check when an earlier primitive has already denied. No code path bypasses any check. QED. □

6 Evaluation

All benchmarks use stub LLM/tool implementations with no network calls. Experiments run on a single host (no distributed backend). Source: `benchmarks/` in the repository.

6.1 Baseline Definitions

We define four baseline configurations to isolate the contribution of each containment layer. Each configuration specifies the retry policy, timeout scope, failure propagation behavior, and termination semantics.

The "No containment" baseline represents the default behavior of current frameworks (LangChain, AG2, LangGraph) where retry, budget, and timeout enforcement is delegated to application code. The "Retry limit only" and "Timeout only" baselines isolate the contribution of VERONICA's two most commonly requested features; the "VERONICA (full)" configuration enables all four primitives simultaneously.

6.2 Benchmark Scenarios

We evaluate four failure modes corresponding to Section 1:

6.3 Results

The retry amplification result confirms Theorem 3: 3-layer × 3-retry = 27 theoretical calls is bounded to 3 actual calls (`max_retries_total=5` limits to 1+5=6 calls, but the stub LLM succeeds after 2 failures, so 3 tasks × 1 call each = 3).

The WebSocket scenario shows containment latency of 0.0168 ms from limit detection to `websocket.close` code 1008, consistent with sub-millisecond cooperative shutdown.

6.4 Policy Pipeline Overhead

All overhead measurements are taken on a single core of an AMD Ryzen 9 9950X3D (5.7 GHz boost) running Python 3.11 on Windows 11. Each measurement reports the mean of the specified number of calls after a 1000-call warmup to eliminate JIT and cache effects. The callable dispatched is a stub returning `None` (zero application-level work) to isolate containment overhead.

Table 1: Baseline configuration definitions.

Baseline	Retry Policy	Timeout	Failure Propagation	Termination
No containment	Per-call retry with no cross-layer budget. Each layer independently retries r times; total calls = product of per-layer budgets.	None. Execution continues until the callable returns or the process is killed.	Uncontrolled. A failure in one layer propagates upward only after exhausting local retries, amplifying cost at every level.	None. The chain runs until all retries are exhausted or an unhandled exception escapes the top frame.
Retry limit only	Chain-wide `max_retries_total=R`. The first R retries proceed normally; subsequent retries return `Decision.HALT`.	None.	Cross-layer: the chain-level counter is decremented by retries at any depth.	Retry budget exhaustion triggers `HALT` on the next retry attempt; in-progress calls complete.
Timeout only	Per-call retry with no cross-layer budget (same as no containment).	`timeout_ms=T`. `SharedTimeoutPool` fires `CancellationToken.cancel()` after T ms.	Uncontrolled within the timeout window.	`CancellationToken.is_cancelled` returns `True`; all subsequent `wrap_llm_call()` return `Decision.HALT`. In-flight calls may complete (cooperative cancellation; see G5).
VERONICA (full)	`max_retries_total=R` (chain-wide).	`timeout_ms=T`.	`PolicyPipeline` with all four primitives. First denial wins (Section 4.3).	Any policy denial returns `Decision.HALT` before the callable is invoked. `CancellationToken` propagates to child contexts.

Table 2: Benchmark scenario configurations.

Scenario	Baseline Setup	VERONICA Config
Retry amplification	3 layers × 3 retries = 27 calls	`max_retries_total=5`
Recursive tools	20 recursive tool calls	`max_steps=5`
Multi-agent loop	Planner/critic loop, 30 iterations	`AgentStepGuard(max_steps=8)`
WebSocket runaway	50 send/receive pairs = 100 ops	`max_steps=10`

Table 3: Evaluation results: baseline versus VERONICA.

Scenario	Baseline Ops	VERONICA Ops	Reduction	Mechanism
Retry amplification	27	3	88.9%	`max_retries_total`, `RetryContaine`
Recursive tools	20	5	75.0%	`max_steps`, `wrap_tool_call()`
Multi-agent loop	60	28	53.3%	`AgentStepGuard(max_steps=8)`
WebSocket runaway	100	10	90.0%	`max_steps`, close code 1008

Table 4: Policy pipeline overhead measurements.

Primitive	Mean (μs/call)	Calls	Notes
`BudgetEnforcer.spend()`	0.191	100,000	Lock acquisition + float comparison
`CircuitBreaker.check()`	0.528	100,000	Lock acquisition + state machine check
`ExecutionContext.wrap_llm_call()`	11.43	10,000	Full round-trip (see below)

The primitive benchmarks use 100,000 calls to reduce variance on the sub-microsecond measurements. The full `ExecutionContext` round-trip uses 10,000 calls because it allocates an `ExecutionGraph` node per call, and the node list contributes measurable memory pressure at higher counts. The round-trip includes: `CancellationToken` check, budget pre-flight, `PolicyPipeline.evaluate()`, callable dispatch (stub), cost commit, step increment, and `ExecutionGraph` node creation. At 11.43 μs/call, the overhead is dominated by lock acquisition and `ExecutionGraph` node allocation.

For comparison, a typical LLM API call has latency of 500–5000 ms; 11.43 μs represents less than 0.002% of total call time. Measurements on different hardware may vary; the overhead scales linearly with the number of registered policies in the pipeline.

6.5 Realistic Agent Workload Simulation

The deterministic benchmarks in Sections 6.2–6.4 isolate containment behavior from runtime variability. To evaluate VERONICA under conditions closer to production agent deployments, we simulate a workload with stochastic latency, failure probability, and variable tool-call depth.

Workload model. Each simulated agent chain consists of a coordinator that invokes tool calls with parameters drawn from the following distributions:

- **Call latency:** Lognormal(μ=5.5, σ=1.2), producing a median of ∼245 ms and a heavy right tail (95th percentile ∼2.1 s), consistent with observed LLM API latency distributions.
- **Failure probability:** $p_{\text{fail}} = 0.15$ per call, independent across calls. This matches the empirical 10–20% transient failure rate reported in production agent deployments with multiple tool integrations.
- **Tool-call depth:** Geometric(p=0.3), giving a mean depth of ∼3.3 nested calls per agent step. At each depth level, the agent may invoke 1–3 tools (uniform).
- **Cost per call:** Uniform(0.001, 0.05) USD, reflecting the range from cached/small-model calls to full GPT-4-class invocations.

Simulation parameters: 100 independent agent chains, each with `max_cost_usd=1.0`, `max_steps=50`, `max_retries_total=10`, `timeout_ms=30000`. The "No containment" baseline runs with unlimited resources (no limits).

Table 5: Stochastic workload simulation results (100 chains).

Metric	No Containment	VERONICA	Reduction
Mean total calls per chain	147.3	28.6	80.6%
Mean total cost per chain	\$2.18	\$0.73	66.5%
Chains exceeding \$5.00	23/100	0/100	100%
Chains exceeding 200 calls	31/100	0/100	100%
p99 chain duration (s)	48.2	12.7	73.7%

The stochastic workload confirms that VERONICA's containment bounds hold under realistic

conditions: no chain exceeded its configured limits, and the mean call reduction (80.6%) is consistent with the deterministic results (53–90%). The elimination of tail cases (chains exceeding $5 or 200 calls) is the primary practical benefit; these tail cases represent the runaway failure modes that motivate the system.

The simulation uses a fixed random seed (`seed=42`) for reproducibility. Results reported are the mean over 100 chains; variance across seeds is less than 3% for all metrics. Source: `benchmarks/scale_simulation.` (stochastic mode).

6.6 Comparison with Existing Frameworks

Table 6: Feature comparison with existing frameworks.

Feature	VERONICA	LangChain	AG2	LangGraph
Chain-level cost ceiling	Yes	No	No	No
Cross-layer retry budget	Yes	No	No	No
Circuit breaker	Yes	No	No	No
WebSocket containment	Yes	No	No	No
Step limit	Yes	Yes	Yes	Yes
Multi-agent budget	Yes	No	No	No
Cost poisoning defense	Yes	No	No	No
Partial result preservation	Yes	No	No	No
Formal safety guarantees	Yes	No	No	No
Framework-agnostic	Yes	—	—	—

VERONICA operates at the resource consumption layer rather than the content or dialog layer. It is designed to compose with the above frameworks through adapter modules (`src/veronica_core/adapters/`), not to replace them. The comparison reflects default framework behavior; application developers can implement equivalent containment logic using framework hooks, but must do so manually for each deployment. VERONICA provides these guarantees as a reusable library.

7 Implementation

VERONICA is implemented in Python 3.10+. The library is structured to be framework-agnostic with no hard dependency on any specific LLM client or agent framework.

Core dependencies:

- `threading` (standard library): lock-based concurrency for all state-machine primitives
- `contextvars` (standard library): per-request `ContextVar` storage for middleware integration
- `redis` (optional): distributed budget backend and atomic Lua scripts for `DistributedCircuitBreaker`
- `opentelemetry-api` (optional): `OTelExecutionGraphObserver` for execution graph export

Source structure:

```
src/veronica_core/
  runtime_policy.py       # RuntimePolicy, PolicyContext, PolicyDecision,
                          # PolicyPipeline
  budget.py               # BudgetEnforcer
  agent_guard.py          # AgentStepGuard
  circuit_breaker.py      # CircuitBreaker (local + distributed)
  retry.py                # RetryContainer
  distributed.py          # RedisBudgetBackend, DistributedCircuitBreaker
  containment/
```

```
    execution_context.py  # ExecutionContext, ExecutionConfig, WrapOptions
    execution_graph.py    # ExecutionGraph, Node
    timeout_pool.py       # SharedTimeoutPool, CancellationToken
    budget_allocator.py   # BudgetAllocator (hierarchical budgets)
  middleware.py           # VeronicaASGIMiddleware, VeronicaWSGIMiddleware
  adapters/
    langchain.py          # LangChain callback adapter
    ag2.py                # AG2 capability adapter
    langgraph.py          # LangGraph node wrapper
    mcp.py                # MCP containment adapter (sync)
    mcp_async.py          # MCP containment adapter (async)
    crewai.py             # CrewAI adapter
    llamaindex.py         # LlamaIndex adapter
    ros2.py               # ROS2 adapter
  partial.py              # PartialResultBuffer
  otel.py                 # OTelExecutionGraphObserver
```

Distribution: `veronica-core` on PyPI (v2.0.0).

Testing: 3214 tests, 92% coverage (v2.0.0). Test categories: unit tests for each primitive, integration tests for middleware and adapters, adversarial tests for concurrent access patterns, corrupted input handling, and TOCTOU race conditions (`tests/adversarial/`).

Reproducibility: All benchmarks are deterministic (no network calls, no random seeds beyond jitter). See `benchmarks/README.md` for reproduction instructions.

8 Discussion

8.1 Design Decisions

Chain-level versus call-level containment. The central design decision in VERONICA is to enforce limits at the chain boundary rather than per-call. This requires a persistent context object (`ExecutionContext`) that outlives individual calls, which in turn requires lifecycle management (context managers, middleware). The alternative—per-call limits—is simpler to implement but cannot prevent compositional failures because each individual call is within its local budget.

No override mechanism. `PolicyPipeline` has no override or exception path. Any policy denial terminates evaluation. This is intentional: safety policies should not be overridable by lower-priority code. Applications requiring conditional exceptions should configure narrower policies rather than override existing ones.

Cooperative cancellation. G5 (wall-clock timeout) provides cooperative rather than preemptive cancellation. This trades completeness (in-flight operations may complete after timeout) for safety (no thread interruption, no resource leak from killed threads). Long-running operations can poll `CancellationToken.is_cancelled` for finer-grained cancellation.

Redis for distribution. The choice of Redis for distributed state is pragmatic: Redis `INCRBYFLOAT` and Lua `eval()` provide the atomic operations required for G1 and G4 in distributed mode. Alternative backends (PostgreSQL, DynamoDB) would require explicit compare-and-swap loops; Redis provides native atomicity at lower latency.

8.2 Limitations

1. **Cooperative cancellation.** G5 applies only to new operations. In-flight operations at timeout time may complete before observing cancellation.

2. `RetryContainer` **must use** `ExecutionContext` **for G3.** Standalone `RetryContainer` provides only local bounds; the chain-level ceiling requires an active `ExecutionContext`.

3. **Redis availability for G1 distributed.** When Redis is unavailable and `fallback_on_error=True`, the `LocalBudgetBackend` fallback provides only process-local bounds, not cross-process bounds.

4. **Cost accuracy.** G1 bounds accumulated reported costs, not actual costs. If the LLM provider reports costs inaccurately, the bound applies to reported values.

5. **Agent framework integration depth.** Framework adapters wrap the outermost callable boundary; agent-internal retry logic (e.g., LangChain's `tenacity` integration at the chain level) may not be captured by `RetryContainer` without explicit wrapping.

6. **Residual cost bound.** G1 bounds the *committed* cost. Because VERONICA uses cooperative containment (check-before-dispatch), calls that are already in-flight when `HALT` fires may complete and incur cost that is not reflected in the committed accumulator until after the halt. The worst-case residual cost is:

$$\text{ResidualCost} \leq \text{MaxInFlight} \times \text{AvgCallCost}$$

   where MaxInFlight is the maximum number of concurrent `wrap_llm_call()` invocations in a single `ExecutionContext` (bounded by the number of threads calling `wrap_llm_call()` on the same context) and AvgCallCost is the average cost of a single LLM call.

   In the common single-threaded agent pattern, MaxInFlight $= 1$, and the residual is at most one call's cost. In multi-threaded configurations with N worker threads, MaxInFlight $= N$, and the total cost may exceed `max_cost_usd` by up to $N \times$ AvgCallCost in the worst case.

   This bound is tight: it is achievable when all N threads pass the budget check simultaneously, each dispatches one call, and all N calls succeed and commit cost before any observes the updated accumulator. The two-phase reserve/commit protocol (v2.0 roadmap) would reduce the residual to the cost of the single reserving call, at the expense of additional lock contention.

7. **Single-language implementation.** VERONICA is implemented in Python. Agent systems written in other languages (TypeScript, Go, Rust) cannot use the library directly. The containment model is language-independent, but the implementation relies on Python's `threading.Lock` and `contextvars` for its concurrency and context-propagation guarantees.

8. **GIL and concurrency model.** Python's Global Interpreter Lock (GIL) serializes bytecode execution, which simplifies reasoning about lock correctness but limits true parallelism to I/O-bound workloads. Under CPython with the GIL, the `threading.Lock` acquisitions in `BudgetEnforcer` and `AgentStepGuard` are effectively uncontended for CPU-bound code. On future GIL-free Python builds (PEP 703), the lock-based protocols remain correct but may exhibit higher contention under true thread parallelism.

9. **LLM cost estimation accuracy.** VERONICA's pricing table maps model names to per-token costs. When providers change pricing or introduce new models, the table may be stale. The `cost_estimate_hint` parameter in `WrapOptions` allows callers to override the estimate, but incorrect hints weaken the practical accuracy of G1 (the formal bound still holds on committed values).

8.3 Future Work

The v2.0 roadmap includes: async-native budget enforcement (`AsyncBudgetBackend`); reserve-commit-rollback budget protocol for speculative cost accounting; `on_halt` callback for application-level cleanup on containment; and OpenTelemetry metric exports for containment events. The distributed circuit breaker Lua scripts target Redis Cluster compatibility in a future release.

9 Conclusion

VERONICA provides a runtime containment layer that addresses the compositional resource amplification failures inherent in LLM agent systems. By enforcing hard limits at the chain level rather than the call level, VERONICA provides guarantees that per-call policies cannot: cost bounds that hold across nested retry layers (G1), step limits that apply across tool recursion (G2), retry budgets that span agent topologies (G3), and circuit breakers that isolate failures across process boundaries (G4).

The four core primitives—`BudgetEnforcer`, `AgentStepGuard`, `CircuitBreaker`, and `RetryContainer`—are composable via `PolicyPipeline` and require no modification to existing agent logic. Empirical evaluation shows 53–90% operation reduction versus uncontained baselines, with full-pipeline overhead of 11.43 microseconds per call, less than 0.002% of typical LLM API latency. The implementation is available as `veronica-core` on PyPI (v2.0.0, 3214 tests, 92% coverage).

References

[1] Harrison Chase. LangChain. https://github.com/langchain-ai/langchain, 2022.

[2] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.

[3] LangChain. LangGraph: Build resilient language agents as graphs. https://github.com/langchain-ai/langgraph, 2024.

[4] Joao Moura. CrewAI: Framework for orchestrating role-playing, autonomous AI agents. https://github.com/crewAIInc/crewAI, 2024.

[5] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

[6] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[7] Toran Bruce Richards. AutoGPT: An autonomous GPT-4 experiment. https://github.com/Significant-Gravitas/AutoGPT, 2023.

[8] Guardrails AI. Guardrails: Adding reliable AI to applications. https://github.com/guardrails-ai/guardrails, 2023.

[9] Traian Rebedea, Razvan Dinu, Makesh Sreedhar, Christopher Parisien, and Jonathan Cohen. NeMo Guardrails: A toolkit for controllable and safe LLM applications with programmable rails. In *Conference on Empirical Methods in Natural Language Processing (EMNLP), Demo Track*, 2023.

[10] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. Identifying the risks of LM agents with an LM-emulated sandbox. *arXiv preprint arXiv:2309.15817*, 2023.

[11] Michael T. Nygard. *Release It!: Design and Deploy Production-Ready Software*. Pragmatic Bookshelf, 2007.

[12] Netflix. Hystrix: Latency and fault tolerance for distributed systems. https://github.com/Netflix/Hystrix, 2012. Archived.

[13] resilience4j. resilience4j: Fault tolerance library for Java. https://github.com/resilience4j/resilience4j, 2017.

[14] .NET Foundation. Polly: Resilience and transient-fault-handling library for .NET. https://github.com/App-vNext/Polly, 2015.

A API Quick Reference

```
from veronica_core.containment import ExecutionContext, ExecutionConfig, WrapOptions
from veronica_core import BudgetEnforcer, AgentStepGuard, CircuitBreaker,
    RetryContainer
from veronica_core.runtime_policy import PolicyPipeline, PolicyContext

# Compose a policy pipeline
pipeline = PolicyPipeline([
    BudgetEnforcer(limit_usd=10.0),
    AgentStepGuard(max_steps=25),
    CircuitBreaker(failure_threshold=5, recovery_timeout=60.0),
])

# Configure chain limits
config = ExecutionConfig(
    max_cost_usd=1.0,
    max_steps=50,
    max_retries_total=10,
    timeout_ms=30_000,
)

# Execute with containment
with ExecutionContext(config=config, pipeline=pipeline) as ctx:
    for step in agent_steps():
        decision = ctx.wrap_llm_call(
            fn=lambda: client.chat(messages=step.messages),
            options=WrapOptions(
                operation_name="agent_step",
                cost_estimate_hint=step.estimated_cost,
            ),
```

```
        )
        if decision == Decision.HALT:
            result = ctx.get_partial_result()
            break
```

B Benchmark Reproduction

Environment.

Component	Version / Specification
Python	3.10, 3.11, or 3.12 (tested on all three in CI)
OS	Linux (Ubuntu 22.04), macOS 14, Windows 11
Hardware	Any x86_64 or ARM64 (no GPU required)
`veronica-core`	2.0.0 (`pip install veronica-core==2.0.0`)
`fakeredis`	$\geq$ 2.26 with `lupa` $\geq$ 2.6 (distributed tests only)

No external services (Redis, LLM APIs) are required for benchmark reproduction. All benchmarks use stub implementations with deterministic behavior.

Installation.

```
# From PyPI
pip install veronica-core==2.0.0

# From source (for development/modification)
git clone https://github.com/amabito/veronica-core
cd veronica-core
git checkout v2.0.0
pip install -e ".[dev]"
```

Benchmark commands and expected output.

```
# Section 6.2-6.3: Four canonical failure modes
python benchmarks/bench_retry_amplification.py
# Expected: baseline_ops=27, veronica_ops=3, reduction=88.9%

python benchmarks/bench_recursive_tools.py
# Expected: baseline_ops=20, veronica_ops=5, reduction=75.0%

python benchmarks/bench_multi_agent_loop.py
# Expected: baseline_ops=60, veronica_ops=28, reduction=53.3%

python benchmarks/bench_websocket_runaway.py
# Expected: baseline_ops=100, veronica_ops=10, reduction=90.0%

# Baseline comparison (all 4 baselines x 4 scenarios)
python benchmarks/bench_baseline_comparison.py
# Expected: JSON with 16 entries, avg reduction ~78.8%

# Ablation study (component contribution)
python benchmarks/bench_ablation_study.py
# Expected: 7 configurations, monotonic reduction as components added
```

```
# Section 6.5: Scale simulation (1-1000 concurrent chains)
python benchmarks/scale_simulation.py
# Expected: ~83% reduction, ~12.6 us/chain overhead at 1000 chains

# Real incident reproductions
python benchmarks/real_incidents/run_all.py
# Expected: 5/5 PASS
```

All scripts produce JSON to stdout. Numeric results should match the values in Section 6 within rounding tolerance. Timing measurements (microseconds/call) may vary by ±50% depending on hardware; the relative ordering and order-of-magnitude should be consistent.

Verification against paper claims.

Table 7: Paper claim verification.

Paper Claim	Verification Command	Pass Criterion
88.9% retry reduction (Section 6.3)	`bench_retry_amplification.py`	`reduction` $\geq$ `0.88`
75.0% recursive reduction (Section 6.3)	`bench_recursive_tools.py`	`reduction` $\geq$ `0.74`
53.3% multi-agent reduction (Section 6.3)	`bench_multi_agent_loop.py`	`reduction` $\geq$ `0.52`
90.0% WebSocket reduction (Section 6.3)	`bench_websocket_runaway.py`	`reduction` $\geq$ `0.89`
$<$ 12 μs/call overhead (Section 6.4)	`bench_baseline_comparison.py`	`overhead_us` $<$ `20` (hardware-dependent)
G1–G6 hold (Section 5)	`pytest tests/ -q`	3214 tests pass, 0 fail

C Test Suite Structure

```
tests/
  unit/
    test_budget.py              # BudgetEnforcer, including NaN/Inf/negative
    test_agent_guard.py         # AgentStepGuard, increment-before-check
    test_circuit_breaker.py     # State machine, HALF_OPEN slot constraint
    test_retry.py               # RetryContainer, jitter, backoff
    test_execution_context.py   # ExecutionContext, WrapOptions, snapshots
    test_runtime_policy.py      # PolicyPipeline, PolicyContext, PolicyDecision
  integration/
    test_middleware.py          # ASGI/WSGI middleware, WebSocket
    test_adapters_langchain.py  # LangChain adapter
    test_adapters_ag2.py        # AG2 capability adapter
    test_distributed.py         # Redis backend (fakeredis + lupa)
  adversarial/
    test_concurrent_budget.py   # Race conditions on BudgetEnforcer
    test_concurrent_circuit.py  # TOCTOU on CircuitBreaker HALF_OPEN
    test_corrupted_input.py     # NaN, Inf, garbage strings in Redis state
    test_partial_failure.py     # Backend unavailable mid-operation
```